ARCHITECTURE REVIEW

Amazon 智能竞品分析系统 数据流与架构分析

前端 · 后端 · Agent 三层的端到端数据通路、核心函数、核心模块与接口实现位置

项目 Analysis System **分支** master **日期** 2026-10-26 **版本** v1.0

目录

1. 一句话结论
2. 分层与核心模块
3. 端到端数据流
4. 前端核心模块
5. 后端核心模块
6. Agent 层核心模块
7. 前后端接口实现文件
8. 数据形态演进
9. 风险清单与修复建议
10. 附：文件索引

01 一句话结论

一次分析任务的完整链路是：

```
Vue 组件 → frontend/src/api/analysis.js（fetch 封装）
         → Vite 代理 /api → FastAPI（backend/app/main.py 路由）
         → analysis_service.py（内存任务表 + 后台线程）
         → AnalysisWorkflow（3 个 Task 串联）
         → Agent（LLM + 工具循环）+ 竞品发现 + 报告生成
         → reports/{task_id}.md
         → 前端 2 秒轮询 → GET report → markdown-it 渲染
```

核心判断

这是一个**「提交任务 → 后台线程执行 → 前端轮询 → 拉取报告」**的异步任务模型。 HTTP 层只做三件事：创建任务、查状态、取报告；真正的业务全部发生在后端进程内的线程与工作流中， 任务状态保存在内存字典，报告以 Markdown 文件落盘。

前后端交互的接口层文件

前端唯一出口	`frontend/src/api/analysis.js`
后端唯一入口	`backend/app/main.py`（`routers.py` 为空文件，路由未拆分）

请求/响应契约	`backend/app/schemas.py`
异步任务实现	`backend/app/analysis_service.py`
开发期连接桥梁	`frontend/vite.config.js`（`/api` → `127.0.0.1:8000`）

02 分层与核心模块

层	目录	职责	运行时载体
前端	`frontend/src`	输入、状态机、轮询、Markdown 渲染	浏览器（Vite dev server）
后端 / 接口层	`backend/app`	HTTP 路由、任务表、后台线程	FastAPI + uvicorn
工作流层	`app/workflow`	任务串联与数据传递	后台线程
Agent 层	`app/agent`	竞品发现、LLM 工具循环、结果校验	线程池
工具层	`app/tools`	搜索、网页抓取、评论采集、知识库	同步函数
LLM / RAG 层	`app/llm`、`app/rag`	统一 OpenAI 兼容客户端、本地向量检索	外部 API
输出层	`app/reporter`	组装 Markdown、落盘	本地文件

03 端到端数据流

3.1 总览

浏览器 Vue 3 · ProductForm / AnalysisStatus / AnalysisView

▼ fetch POST /api/analysis

Vite 代理 /api → http://127.0.0.1:8000（vite.config.js）

▼ HTTP / JSON

FastAPI backend/app/main.py 路由 + schemas.py 校验

▼ create_analysis_task()

任务管理 analysis_service.py：内存任务表 + daemon Thread

▼ 后台线程执行

工作流 AnalysisWorkflow：Task1 发现 → Task2 分析 → Task3 报告

▼ Agent + LLM + Tools

输出 reports/{task_id}.md

▲ 前端每 2 秒轮询 GET /api/analysis/{id}

回传 GET /api/analysis/{id}/report → Markdown 原文 → markdown-it + DOMPurify

3.2 阶段一：创建任务（HTTP 同步返回）

```
productForm.vue:18  handleSubmit()
  └ emit("submit", product)
      └ AnalysisView.vue:109  startAnalysis(product)
          └ api/analysis.js:3  createAnalysis()
              POST /api/analysis  { target_product: "..." }
                  ↓  Vite proxy（vite.config.js:8-13）→ 127.0.0.1:8000
backend/app/main.py:30  create_analysis(request: AnalysisRequest)
  ├ schemas.py:2  AnalysisRequest 校验（min_length=1）
  └ analysis_service.py:68  create_analysis_task(target_product)
      ├ analysis_tasks[task_id] = { status: "pending", ... }
      └ Thread(target=run_analysis, daemon=True).start()
返回 AnalysisResponse { task_id, status:"pending", message, target_product }
```

关键点

HTTP 请求**立即返回 200**，真正的分析在后台线程执行，前端依靠轮询获取结果。

3.3 阶段二：后台工作流（异步）

`analysis_service.py:13 run_analysis` 先把状态置为 `running`，然后分两条路：

Mock 分支（默认走这条）

```
MOCK_MODE = os.getenv("MOCK_MODE", "true").lower() == "true"   # analysis_service.py:11
→ 直接拼一段假 Markdown → save_report(markdown, task_id) → reports/{task_id}.md
```

真实分支

```
analysis_service.py:39  AnalysisWorkflow()
    └ tasks.py:137  create_workflow_tasks()  →  [Task, Task, Task]
    └ analysis_workflow.py:13  run(input_data)
          input_data = { task_id, target_product }
```



Task 1 — discovery_task（tasks.py:10）

```
find_target_asin(target_product)                    competitor_verifier.py:36
  └ Tavily 搜索 "xxx site:amazon.com product"
  └ 从 URL 正则提取 ASIN、校验容量
  → { asin, title, url }

warmup_product_reviews(target_asin)                 product_review_tool.py:18
  └ 提交到独立线程池（不阻塞），供后面分析复用

run_discovery(target_product)                       competitor_discovery.py:121
  ├ build_search_queries()                          :6   生成 3 条“找替代品”搜索词
  ├ discover_candidates()                           :22  3 线程并发 Tavily 搜索
  │    过滤：必须是 /dp/ 或 /gp/product/、容量匹配、排除自身品牌、URL 去重，最多 5 个
  ├ extract_competitors(target, results)            discovery_parser.py:11
  │    └ LLM 从搜索结果 JSON 提取候选（temperature=0）
  │    └ 校验：source_url 必须来自输入结果、标题需匹配、ASIN 与 URL 一致
  └ verify_candidate(candidate) ×≤5，3 线程并发     competitor_verifier.py:104
       └ 搜独立详情页 → read_webpage → extract_product（LLM） → verify_product_specs
       → 回填 asin / source_url / verification_status

data["competitors"] = 前 3 个 { name, asin, url }
返回同一个 dict（新增 target_asin、competitors）
```

Task 2 — analysis_task（tasks.py:54）

```
ThreadPoolExecutor(max_workers=3) 并发 analyze_one(comp)
  └ agent.py:70  competitor_agent(target, comp.name, target_asin, comp_asin, comp_url)
      messages = [system_prompt, user 描述]
      ├ 只放行 3 个工具：search_knowledge / search_web / analyze_product_reviews（agent.py:99）
      ├ 首轮强制 tool_choice = search_knowledge（agent.py:119）
      ├ 循环最多 3 轮（agent.py:126）
      │   ├ 有 tool_calls → 执行工具 → 结果 json.dumps 追加为 role=tool
      │   │    search_knowledge      → RAG 检索（knowledge_tool.py:2 → rag/retriever.py:17）
      │   │    search_web            → 有 asin 直读详情页；否则 Tavily + 抓页 + LLM 提取（agent.py:196）
      │   │    analyze_product_reviews → 评论单飞缓存（product_review_tool.py:36）
      │   └ 无 tool_calls → _parse_final_result()
      │       └ 去 ``` 包裹 → json.loads → CompetitorAnalysis(**data)
      │       → 校验目标 ASIN 一致、标题非空
      │       → attach_review_analysis()（result_processor.py:3）合并评论分析
      └ 第 3 轮追加 system 消息并 tool_choice="none"，强制出 JSON
  结果包装为 { competitor_name, competitor_asin, competitor_url, analysis }
过滤掉 error 项；全部失败则 RuntimeError
返回 { task_id, target_product, analyses: [...] }
```

Task 3 — report_task（tasks.py:121）

```
generate_markdown(result)              reporter/markdown.py:94
  ├ dict → CompetitorAnalysis 二次校验，非法项跳过
  ├ 目标产品区（format_product + format_review_analysis）
  ├ 每个竞品：详情 + 核心指标对比表 + 价格差 + 总结
  └ 数据说明
save_report(markdown, task_id)         reporter/markdown.py:149
  → Path("reports") / f"{task_id}.md"
  → 返回 Path
```

`AnalysisWorkflow.run`（`analysis_workflow.py:14-24`）把每一步的返回值作为下一步输入；任一步异常则中断，并返回 `{"error": ...}`。

run_analysis 收到的返回值	任务状态
`Path` 且文件存在	`completed`，`record["result"] = 路径`
`{"error": ...}`	`failed`，`record["error"] = ...`
非路径 / 文件不存在	`failed`
抛异常（`analysis_service.py:64`）	`failed`

3.4 阶段三：前端轮询（3 个接口中的第 2 个）

```
AnalysisView.vue:70  pollStatus(id, version)
  每 2 秒  GET /api/analysis/{task_id}
  → backend main.py:52  get_analysis_status
  → { task_id, target_product, status, error }

pending / running → 继续 setTimeout 2000
completed         → getAnalysisReport()
failed            → 显示 error
```

3.5 阶段四：取报告并渲染

```
API  GET /api/analysis/{task_id}/report
  main.py:72  get_analysis_report
  ├ 任务不存在 → 404
  ├ failed    → 409
  ├ 未完成    → 409
  └ completed → path.read_text() → { task_id, status:"completed", report: "..." }

前端 AnalysisView.vue:29
  md.render(report) → DOMPurify.sanitize(html) → v-html
下载：analysis.js 返回原文 → Blob 下载 .md
```

04 前端核心模块

文件	核心函数 / 职责
`frontend/src/main.js`	挂载 Vue 应用
`frontend/src/App.vue`	唯一入口，渲染 `AnalysisView`
`frontend/src/views/AnalysisView.vue`	**状态中心**：`startAnalysis`(:109)、`pollStatus`(:70)、`downloadReport`(:35)、`renderedReport`(:29)
`frontend/src/components/productForm.vue`	输入 + 校验 + `emit("submit")`(:18)
`frontend/src/components/AnalysisStatus.vue`	纯展示组件，按 `taskStatus` 显示文案
`frontend/src/api/analysis.js`	**唯一网络层**：3 个 fetch 函数
`frontend/vite.config.js`	`/api` 代理到 `127.0.0.1:8000`

前端状态机：`"" → pending → running → completed | failed`。
并发保护：`requestVersion`（`AnalysisView.vue:60,120,131`）防止旧请求覆盖新结果；`onUnmounted` 停止轮询。

05 后端核心模块

文件	核心函数	说明
`backend/app/main.py`	`create_analysis` (:30)、`get_analysis_status` (:52)、`get_analysis_report` (:72)、`health_check` (:21)	**全部路由都在这一个文件**
`backend/app/schemas.py`	`AnalysisRequest` 、`AnalysisResponse`	唯一使用 Pydantic 的接口契约
`backend/app/analysis_service.py`	`create_analysis_task` (:68)、`run_analysis` (:13)、`analysis_tasks` (:12)	内存任务表 + daemon 线程
`backend/app/routers.py` / `services.py`	—	空文件（预留，未实现）

> **注意**
>
> 任务状态存在**进程内全局字典** `analysis_tasks`，没有数据库、没有磁盘持久化。

06 Agent 层核心模块

文件	核心函数	作用
`app/agent/agent.py`	`competitor_agent` (:70)、`_parse_final_result` (:22)	单竞品 LLM 工具循环（最多 3 轮）+ 结果解析校验
`app/agent/competitor_discovery.py`	`build_search_queries` (:6)、`discover_candidates` (:22)、`run_discovery` (:121)	竞品发现主流程
`app/agent/discovery_parser.py`	`extract_competitors` (:11)	LLM 从搜索结果提取候选 + 本地规则校验
`app/agent/competitor_verifier.py`	`extract_asin` (:13)、`find_target_asin` (:36)、`verify_candidate` (:104)	ASIN 解析、目标商品定位、候选详情页核验
`app/agent/extractor.py`	`extract_product` (:14)	LLM 从网页文本抽取结构化商品（带缓存、8000 字截断）
`app/agent/prompts.py`	`system_prompt` 、`competitor_discovery_prompt`	全部提示词与约束规则

文件	核心函数	作用
`app/agent/result_processor.py`	`attach_review_analysis` (:3)	把评论分析合并进 `CompetitorAnalysis`
`app/agent/verification_status.py`	`verify_product_specs`	参数核验
`app/llm/client.py`	`client`	**全项目唯一 LLM 客户端**（OpenAI 兼容）
`app/tools/schema.py`	`tools`	工具 JSON Schema（4 个，Agent 只用 3 个）
`app/tools/web_search.py`	`search_web` (:15)	Tavily + 内存缓存
`app/tools/web_reader.py`	`read_webpage` (:17)	requests + BeautifulSoup + 缓存 + 3 次重试
`app/tools/knowledge_tool.py`	`search_knowledge` (:2)	RAG 检索封装
`app/tools/product_review_tool.py`	`warmup_product_reviews` (:18)、`analyze_product_reviews_tool` (:36)	评论分析预热与 single-flight
`app/analyzers/product_review_pipeline.py`	`analyze_product_reviews` (:12)	BrightData 采集 → LLM 情感分析
`app/analyzers/user_analyzer.py`	`analyze_review_sentiment` (:66)	评论痛点 / 优点 / 建议分析
`app/tools/brightdata_reviews.py`	`collect_amazon_reviews` (:28)	BrightData 数据集采集
`app/workflow/analysis_workflow.py`	`AnalysisWorkflow.run` (:13)	任务串联执行器
`app/workflow/tasks.py`	`discovery_task` (:10)、`analysis_task` (:54)、`report_task` (:121)、`create_workflow_tasks` (:137)	三阶段任务定义
`app/reporter/markdown.py`	`generate_markdown` (:94)、`save_report` (:149)	报告模板与落盘
`app/models/*`	`CompetitorAnalysis`、`CompetitorDiscovery`、`ProductExtraction`、`ReviewAnalysis`	全链路数据契约

07 前后端接口实现文件（重点）

前后端之间只有 **3 个 HTTP 接口**，全部定义在 `backend/app/main.py`：

POST `/api/analysis`

用途	创建分析任务，立即返回 task_id
请求体	`{ "target_product": "HUADISK 256GB SSD" }` （`AnalysisRequest`）
响应体	`{ task_id, status:"pending", message, target_product }` （`AnalysisResponse`）
处理函数	`main.py:31 create_analysis` → `analysis_service.py:68 create_analysis_task`
前端调用	`api/analysis.js:3 createAnalysis`

GET `/api/analysis/{task_id}`

用途	查询任务状态（前端每 2 秒轮询）
响应体	`{ task_id, target_product, status, error }`
状态值	`pending` / `running` / `completed` / `failed`
处理函数	`main.py:53 get_analysis_status` （直接读 `analysis_tasks`）
前端调用	`api/analysis.js:26 getAnalysisStatus`

GET `/api/analysis/{task_id}/report`

用途	获取 Markdown 报告全文
响应体	`{ task_id, status:"completed", report: "..." }`
错误码	404 任务不存在 / 409 未完成或失败 / 500 路径无效
处理函数	`main.py:73 get_analysis_report` → `Path.read_text()`
前端调用	`api/analysis.js:42 getAnalysisReport`

另有 `GET /health`（`main.py:21`）用于健康检查，前端未使用。

> **接口契约链路**
>
> `schemas.py` 定义请求/响应模型 → `main.py` 声明 `response_model` 并实现路由 → `api/analysis.js` 以相同的字段名（`target_product`、`task_id`、`status`、`report`）发起请求与解析响应。 三者之间**没有自动生成的类型共享**，字段名是手工对齐的。

08 数据形态演进

阶段	数据结构
HTTP 请求	`{ "target_product": "..." }`
任务表记录	`{ task_id, target_product, status, result, error }`

阶段	数据结构
工作流输入	`{ task_id, target_product }`
竞品发现后	`+ target_asin`、`+ competitors: [{ name, asin, url }]`
竞品分析后	`{ task_id, target_product, analyses: [{ competitor_name, competitor_asin, competitor_url, analysis }] }`
`analysis` 字段	`CompetitorAnalysis` Pydantic 对象（或 `{"error": ...}`），序列化后才进入报告
报告	Markdown 字符串
报告接口	`{ task_id, status, report }`
前端渲染	`report` 字符串 → HTML → `DOMPurify.sanitize` → `v-html`

线程模型

执行位置	并发方式	代码
每个任务	1 个 daemon `Thread`	`analysis_service.py:78`
工作流三步	串行	`analysis_workflow.py:15`
竞品分析	`ThreadPoolExecutor(3)`	`tasks.py:98`
竞品发现搜索 / 核验	各 `ThreadPoolExecutor(3)`	`competitor_discovery.py:55`、`discovery_parser.py:141`
评论采集	独立 `ThreadPoolExecutor(2)` + single-flight 缓存	`product_review_tool.py:7`

进程内缓存

所有缓存都在内存中，进程重启即失效：网页搜索（`web_search.py:9`）、网页正文（`web_reader.py:15`）、 商品提取（`extractor.py:3`）、评论分析（`product_review_pipeline.py:9`）、 embedding（`rag/embedding.py:9` 的 `lru_cache`）。

09 风险清单与修复建议

等级	问题	位置	修复建议
阻断	JS 文件里写了 `#` 注释，Vite 语法错误，开发服务器无法启动	`frontend/vite.config.js:4`	改为 `//` 注释或删除（当前未提交改动）
阻断	`<style>` 内使用 `//` 注释，`.success` 规则可能整条失效	`AnalysisStatus.vue:58`	改为 `/* ... */` 或删除
高	默认 MOCK 模式，真实 Agent 不会执行	`analysis_service.py:11`	在 `.env` 显式设置 `MOCK_MODE=false`；或把默认值改为 `false`
高	没有 CORS 中间件，生产构建跨域会被浏览器拦截	`backend/app/main.py`	加 `CORSMiddleware`，或由 Nginx 反代同源部署
高	任务状态是进程内字典，多 worker / 重启即丢失，且长期运行内存增长	`analysis_service.py:12`	迁移到 Redis / 数据库，并用 Celery / RQ 等任务队列
中	报告目录使用相对路径，依赖启动目录	`reporter/markdown.py:152`	基于项目根目录拼绝对路径
中	根目录 `main.py` 导入了已不存在的 `workflow_tasks`	`main.py:4`	改用 `create_workflow_tasks()`，或删除旧入口
中	导入 `ProductForm.vue`，实际文件为 `productForm.vue`，Linux 构建会失败	`AnalysisView.vue:6`	统一文件名大小写
中	报告未标注数据来源，与架构文档要求不一致	`reporter/markdown.py:94`	在 `CompetitorAnalysis` 增加 source 字段，并在报告中输出
中	`requirements.txt` 缺少 `fastapi`、`uvicorn`、`httpx2` 等依赖	`requirements.txt`	补全依赖或在 pyproject 中声明
低	大量空占位模块（`routers.py`、`services.py`、`report_generator.py`、`positioning_analyzer.py`、`trend_analyzer.py`）	`backend/app`、`app`	实现或删除，避免误导
低	`discover_competitors` 工具已定义但不在 Agent 的 `allowed_tools` 中，`competitor_tool.py` 也未被调用	`app/tools/schema.py:47`、`app/tools/competitor_tool.py`	明确是否要走“Agent 自主发现”，否则删除

10 附：文件索引

前端

```
frontend/
├── index.html
├── vite.config.js                  # /api 代理
└── src/
    ├── main.js                     # createApp
    ├── App.vue                     # 渲染 AnalysisView
    ├── api/analysis.js             # ★ 前后端交互唯一出口
    ├── views/AnalysisView.vue      # ★ 状态机 / 轮询 / 渲染
    └── components/
        ├── productForm.vue         # 输入
        └── AnalysisStatus.vue      # 状态展示
```

后端

```
backend/app/
├── main.py                         # ★ 全部 HTTP 路由
├── schemas.py                      # ★ 请求 / 响应契约
├── analysis_service.py             # ★ 任务表 + 后台线程
├── routers.py                      # 空
└── services.py                     # 空
```

Agent / 工作流 / 工具

```
app/
├── agent/          agent.py · competitor_discovery.py · discovery_parser.py
│                   competitor_verifier.py · extractor.py · prompts.py
│                   result_processor.py · verification_status.py
├── workflow/       analysis_workflow.py · task.py · tasks.py
├── tools/          schema.py · web_search.py · web_reader.py
│                   knowledge_tool.py · product_review_tool.py
│                   brightdata_reviews.py · competitor_tool.py
├── analyzers/      product_review_pipeline.py · user_analyzer.py
│                   price_analyzer.py · feature_analyzer.py
├── rag/            retriever.py · embedding.py · vector_store.py
├── llm/            client.py
├── models/         competitor.py · discovery.py · extraction.py · review_analysis.py
├── reporter/       markdown.py
└── data/           knowledge/ · reviews/ · vector_store.json
```